AI Slot Machine Experiment Analysis

Team 1738

THE SETUP

What We Built

- Slot machine game (5 reels, casino theme)
- Single-source state (GameState.js)
- Responsive UI (mobile/tablet/laptop)
- Gamification (streaks, jackpot, free spins)
- 30 files, 4200 LOC, fully tested

Tools

- CODEX 5.3 via CLI
- VS, and Git
- 21 iterative prompts over 4 phases

Session 1

Building the Game Logic: Entires 1-3

***there is no demonstration since this session was entirely backend**

What Happened

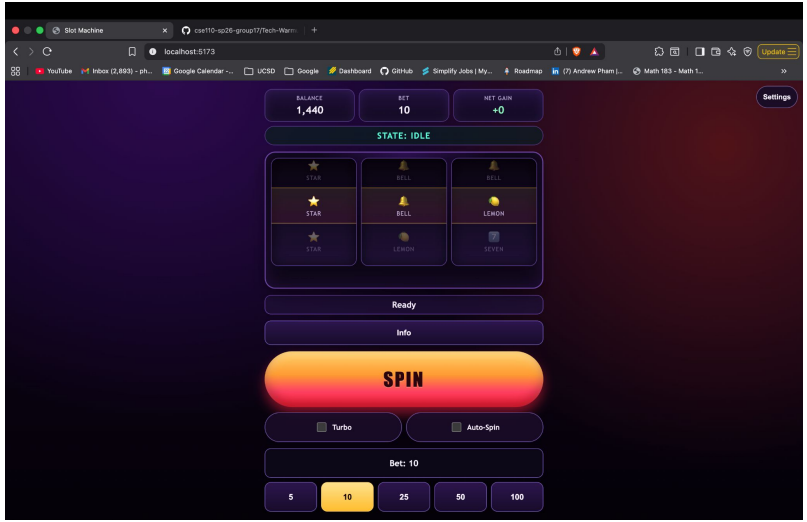
- GameState.js (core game logic)
- Payout tables + RTP calculation
- Win detection + spin evaluation\

Result: All worked first try. Zero bugs.

Lesson: We gave very specific prompts in the beginning and learned that longer prompts dosent necessarily mean it hallucinates. Rather, it works better when focusing only on one aspect of the code, which was backend in this case. It was very bare bone and followed instructions thoroughly, and didn't add anything since we explicitly instructed that.

Session 2

Building the UI:Entires 4-8



What Happened

- Prompt 4: No index.html. Non-runnable.
- Prompt 5: Explicit "must include index.html + package.json + run instructions" → runnable.
- Prompts 6–8: Iterative debugging (animation sync, win payout display, audio overlap)

Result: Took 5 prompts to get working UI. Each fix needed its own prompt.

Why: UI is nondeterministic. Runtime validation is essential.

Session 3

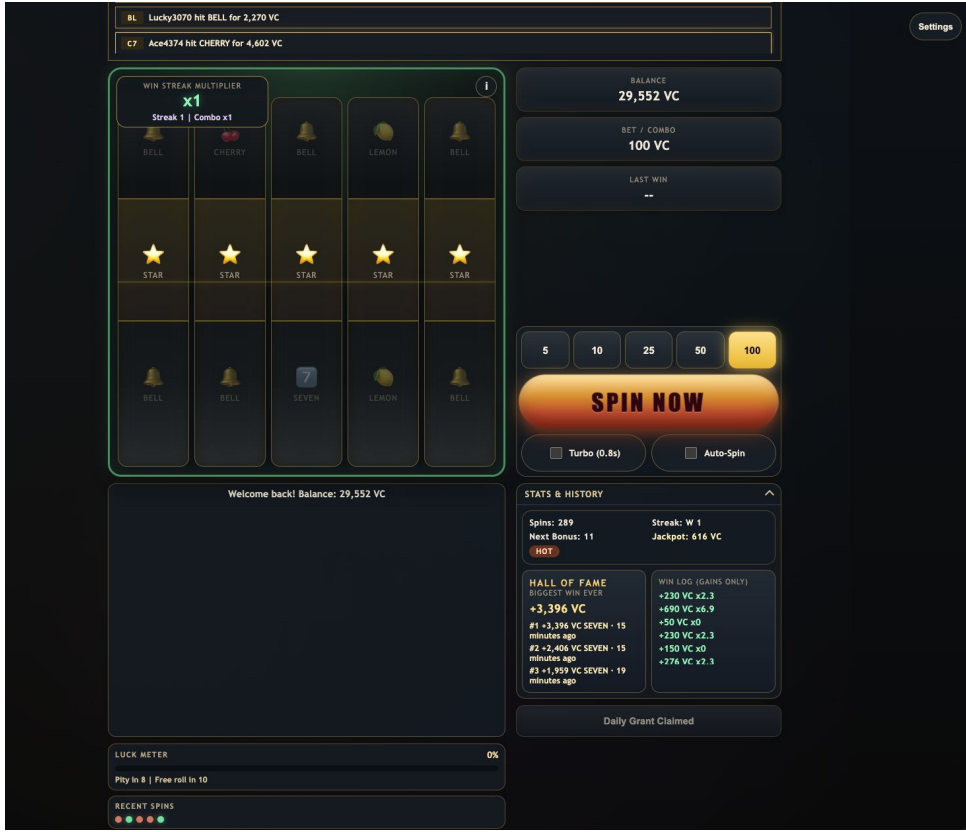
Building the UI: Entires 9-16

What Happened

- Added streaks, jackpot pools, free spins
- RTP was 855% (broke the game)
- Bug: near-miss ternary was a no-op
- Bug: persistence failed on page reload
- Refactored src/ folder (broke imports)

Result: 8 prompts, caught via tests and validation.

Why: Features interact. State management is hard. Tests catch regressions.



Session 4

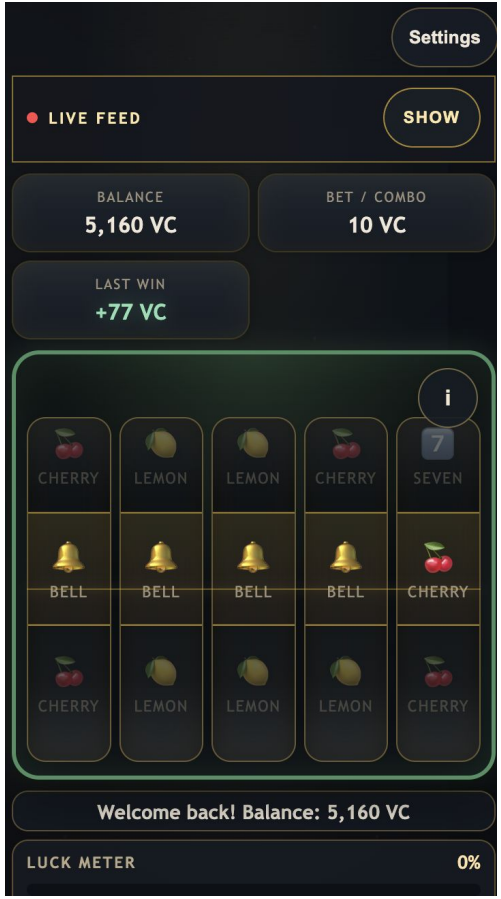
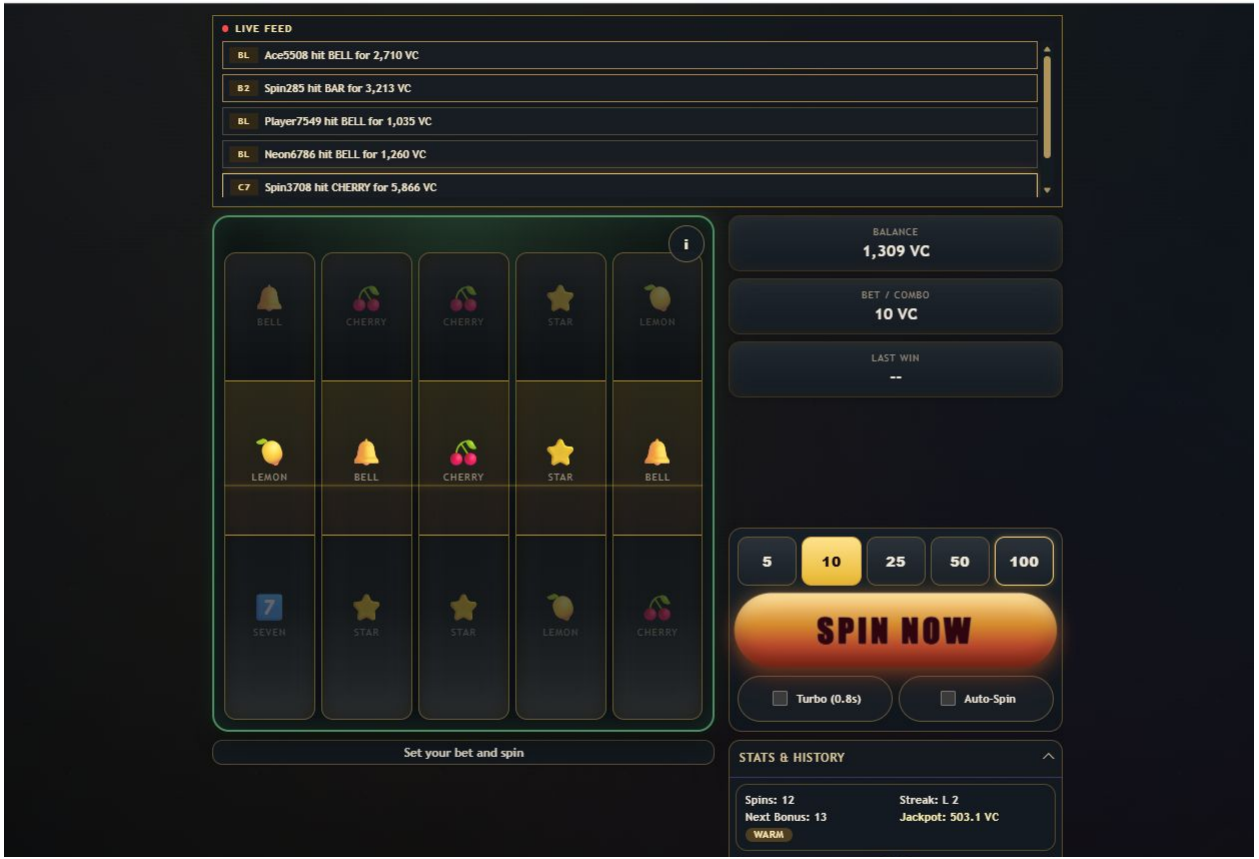
Polish, Gamification & Responsiveness: Entires 17-21

What Happened

- Prompt 17: Mobile responsiveness (explicit 320/768/1024px breakpoints) → no horizontal scroll, 44px+ tap targets
- Prompt 18: Settings button overlaps Live Feed; popup timing unclear → fixed overlap but changed wrong popup
- Prompt 19: Exact numeric specs ("3000ms" streak popup, "+2000ms" win popup) → precise timing verified
- Prompt 20 &21: JSDoc documentation pass, syntax fixation and extended testcases.

Result: Took 4 prompts to polish and document. Numeric specs vs. vague UI requests had different success rates.

Why: Exact constraints (3000ms, +2000ms) prevent ambiguity. Vague UI problems ("popup too long") leave room for AI to guess wrong component.



FINAL PRODUCT